Practicals 1-5: Combined Report

Practical 1: Study and Implementation of Advanced Data Structures in Python

Aim

To study and implement advanced data structures such as Stack, Queue, and Linked List in Python.

Theory

A Stack is a linear data structure that follows the LIFO (Last In First Out) principle, where insertion and deletion happen at one end called the top. The two primary operations are push (insert) and pop (remove), both performed in O(1) time; stacks are used in function call management, expression evaluation, undo mechanisms, and backtracking algorithms. A Queue follows the FIFO (First In First Out) principle, where insertion (enqueue) happens at the rear and deletion (dequeue) at the front; unlike a plain Python list, collections.deque is implemented as a doubly linked list internally, giving O(1) time complexity for operations at both ends instead of O(n) for a list. A Linked List is a linear collection of nodes where each node stores data and a reference (pointer) to the next node, allowing dynamic memory allocation and efficient insertion/deletion (O(1) once the position is known) unlike arrays, which require contiguous memory and O(n) shifting. Variants include singly linked lists (one-directional traversal, used here), doubly linked lists (traversal in both directions), and circular linked lists (last node points back to the first). Together, these three structures form the foundation for more advanced data structures such as trees, graphs, and hash tables.

Algorithm

1. Create an empty list to act as a stack.
2. Push elements onto the stack using append() and pop the last element using pop().
3. Create a deque to act as a queue.
4. Enqueue elements using append() and dequeue using popleft().
5. Create a linked list using nested dictionaries with 'data' and 'next' keys.
6. Traverse the linked list from head to tail, collecting values until next is None.
7. Display the results of all three data structures.

Program

```
stack = []
stack.append(10)
stack.append(20)
stack.append(30)
print("Stack:", stack)
print("Popped:", stack.pop())

from collections import deque
queue = deque()
queue.append("A")
queue.append("B")
queue.append("C")
print("Queue:", list(queue))
print("Dequeued:", queue.popleft())

node = {"data": 1, "next": {"data": 2, "next": {"data": 3, "next": None}}}
values = []
cur = node
while cur:
```

```
        values.append(cur["data"])
        cur = cur["next"]
print("Linked List:", values)
```

Output

```
Stack: [10, 20, 30]
Popped: 30
Queue: ['A', 'B', 'C']
Dequeued: A
Linked List: [1, 2, 3]
```

Result / Conclusion

The stack, queue, and linked list data structures were successfully implemented and their basic operations (push/pop, enqueue/dequeue, traversal) were verified.

Practical 2: Object-Oriented Programming Concepts

Aim

To study and implement Object-Oriented Programming concepts such as classes, objects, inheritance, and method overriding in Python.

Theory

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of objects, which bundle data (attributes) and behavior (methods) together, in contrast to procedural programming which separates data and functions. OOP rests on four main pillars: Encapsulation, which hides internal state and requires interaction through well-defined methods; Abstraction, which exposes only essential features while hiding implementation detail; Inheritance, which allows a class (child/subclass) to acquire properties and methods of another class (parent/superclass), promoting code reuse; and Polymorphism, which allows objects of different classes to be treated through a common interface. A class is a blueprint or template for creating objects, while an object is a concrete instance of a class with its own state. The __init__ method is a constructor, automatically invoked when an object is created, used to initialize instance attributes. Method overriding is a form of runtime polymorphism where a subclass provides a specific implementation of a method already defined in its parent class; when the method is called on a subclass object, Python's dynamic dispatch ensures the overridden version executes, which is demonstrated when Dog and Cat objects both respond differently to the same speak() call.

Algorithm

1. Define a base class Animal with an __init__ method and a speak() method.
2. Define subclasses Dog and Cat that inherit from Animal.
3. Override the speak() method in each subclass to return a specific sound.
4. Create a list of Animal objects containing Dog and Cat instances.
5. Loop through the list and call speak() on each object to demonstrate polymorphism.

Program

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

class Dog(Animal):
    def speak(self):
        return f"{self.name} barks"

class Cat(Animal):
    def speak(self):
        return f"{self.name} meows"

animals = [Dog("Rex"), Cat("Whiskers")]
for a in animals:
    print(a.speak())
```

Output

```
Rex barks
Whiskers meows
```

Result / Conclusion

OOP concepts including class definition, inheritance, and method overriding were successfully implemented and verified using Python.

Practical 3: File Handling and OS Module Operations

Aim

To study and implement file handling and OS module operations such as creating directories, reading/writing files, renaming, and deleting files in Python.

Theory

File handling in Python allows programs to create, read, write, append to, and manipulate files stored on persistent storage using the built-in open() function, which supports modes such as 'w' (write, truncates existing content), 'r' (read), 'a' (append), and 'x' (exclusive creation), along with binary variants like 'wb'/'rb'. Using a 'with' statement (context manager) ensures the file is automatically and safely closed even if an exception occurs, which is why it is preferred over manual open()/close() calls. The os module provides a cross-platform interface to interact with the underlying operating system: os.makedirs() creates a directory (and any missing intermediate directories, with exist_ok=True avoiding an error if it already exists), os.listdir() lists the names of entries in a directory, os.rename() renames or moves a file, os.remove() deletes a file, and os.rmdir() deletes an empty directory. Related modules such as os.path (for path manipulation and existence checks) and shutil (for recursive copy/move/delete of non-empty directories) are commonly used alongside the os module for complete file-system automation.

Algorithm

1. Import the os module.
2. Create a directory using os.makedirs().
3. Open a file in write mode and write text into it, then close it.
4. Open the same file in read mode and display its content.
5. List all files in the directory using os.listdir().
6. Rename the file using os.rename() and list the directory again.
7. Delete the file using os.remove() and delete the directory using os.rmdir().

Program

```
import os
os.makedirs("demo_folder", exist_ok=True)

with open("demo_folder/sample.txt", "w") as f:
    f.write("Hello, File Handling!\n")
    f.write("This is line two.")

with open("demo_folder/sample.txt", "r") as f:
    print("File Content:")
    print(f.read())

print("Files in folder:", os.listdir("demo_folder"))

os.rename("demo_folder/sample.txt", "demo_folder/renamed.txt")
print("After rename:", os.listdir("demo_folder"))

os.remove("demo_folder/renamed.txt")
os.rmdir("demo_folder")
print("Cleanup done")
```

Output

```
File Content:
Hello, File Handling!
This is line two.
Files in folder: ['sample.txt']
```

```
After rename: ['renamed.txt']
Cleanup done
```

Result / Conclusion

File handling and OS module operations (create, read, rename, delete) were successfully performed and verified.

Practical 4: Exception Handling and Fetching Data from a REST API

Aim

To study and implement exception handling in Python while fetching data from a REST API.

Theory

Exception handling in Python is done using try, except, else, and finally blocks: code that might raise a runtime error is placed in the try block, the except block catches and handles specific (or general) exception types so the program does not crash abruptly, an optional else block runs only if no exception occurred, and the finally block always executes regardless of whether an exception was raised, making it ideal for cleanup actions like closing files or connections. Catching specific exception classes (e.g., URLError, TimeoutError) instead of a bare except is considered good practice as it avoids silently hiding unrelated bugs. A REST API (Representational State Transfer) is an architectural style that allows applications to communicate over HTTP using standard methods such as GET, POST, PUT, and DELETE, typically exchanging data in JSON (JavaScript Object Notation) format because it is lightweight and language-independent. The urllib.request module (a built-in alternative to third-party libraries like requests) can be used to send HTTP requests and read server responses, while the json module provides json.loads() to parse a JSON string/bytes into native Python objects (dicts/lists) and json.dumps() to convert Python objects back into a formatted JSON string.

Algorithm

1. Import the json and urllib.request modules.
2. Define the API URL to fetch data from.
3. Use a try block to open the URL and read the response with a timeout.
4. Parse the response using json.loads() and print it in readable format using json.dumps().
5. Use an except block to catch and print any errors during the request.
6. Use a finally block to indicate that the request has completed.

Program

```
import json
import urllib.request

try:
    url = "https://jsonplaceholder.typicode.com/todos/1"
    with urllib.request.urlopen(url, timeout=5) as response:
        data = json.loads(response.read())
        print("API Response:")
        print(json.dumps(data, indent=2))
except Exception as e:
    print("Error fetching data:", e)
finally:
    print("Request completed")
```

Output

```
API Response:
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
Request completed
```

Result / Conclusion

Exception handling was successfully used to safely fetch and parse JSON data from a REST API.

Practical 5: Python Libraries and Basic Web Scraping

Aim

To study and implement basic web scraping using Python libraries requests and BeautifulSoup.

Theory

Web scraping is the process of programmatically extracting information from websites by fetching their raw HTML and parsing out the required data, commonly used for data collection, price monitoring, and research when no structured API is available; it should always be done in compliance with a site's terms of service and robots.txt. The requests library is a popular third-party HTTP client used to send GET/POST requests and retrieve a page's HTML content along with metadata like status codes and headers, and is generally preferred over the lower-level urllib for its simpler, more readable API. BeautifulSoup, from the bs4 package, is an HTML/XML parsing library that builds a navigable parse tree from raw markup and provides convenient methods to search and traverse it, such as tag access via dot notation (soup.title, soup.h1), find() and find_all() to locate elements by tag name, class, or attribute, and .text/.string to extract the readable text content of an element, abstracting away the need to manually write regular expressions against HTML.

Algorithm

1. Import the requests library and BeautifulSoup from bs4.
2. Send a GET request to the target URL to fetch the HTML page.
3. Parse the HTML content using BeautifulSoup with the 'html.parser'.
4. Extract and print the page title using soup.title.string.
5. Extract and print the first heading using soup.h1.text.
6. Find all paragraph tags using soup.find_all('p') and display their count and content.

Program

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"
response = requests.get(url, timeout=5)
soup = BeautifulSoup(response.text, "html.parser")

print("Page Title:", soup.title.string)
print("First Heading:", soup.h1.text)
paragraphs = soup.find_all("p")
print("Number of paragraphs:", len(paragraphs))
print("First paragraph:", paragraphs[0].text)
```

Output

```
Page Title: Example Domain
First Heading: Example Domain
Number of paragraphs: 2
First paragraph: This domain is for use in documentation examples without needing permission. Avoi
```

Result / Conclusion

Basic web scraping was successfully implemented, extracting the title, heading, and paragraph content from a web page.